

SIMATS ENGINEERING
ASSESSMENT TOOL 2: CASE STUDY

CSA 1511- Cloud Computing and Big Data Analytics

Name : NAVADEEP R

Reg No : 192424110

Faculty Incharge : Dr.HEMAVATHI R

CO3 – ASSESSMENT TOOL-2

Code of Conduct:

I Navadeep R (192424110) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Navadeep R

1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

1. Introduction

A Smart Retail Inventory Management System is a cloud-based application designed to help retail chains manage product inventory, sales transactions, store operations, and demand forecasting. The system connects Point of Sale (POS) systems, store devices, and management dashboards to centralized cloud services through APIs.

Cloud computing allows the retail organization to store large amounts of product and transaction data, process analytics,

and forecast future demand without maintaining extensive on-premises infrastructure.

2. Objectives

The main objectives of the system are:

- To maintain product inventory across multiple stores.
- To connect POS systems with centralized cloud services.
- To securely store product and transaction information.
- To analyze sales data using cloud platform services.
- To forecast product demand using analytics and machine learning.
- To provide real-time inventory information through dashboards.
- To improve security using VPNs, firewalls, and API gateways.
- To achieve scalability and availability using cloud infrastructure.

3. Cloud Computing Components

Component	Role in the System
Cloud Infrastructure (IaaS)	Provides virtual machines, networking, computing resources, and infrastructure.
Platform Services (PaaS)	Provides analytics, demand forecasting, databases, and application development services.
Cloud Storage	Stores product data, transaction logs, reports, and analytics datasets.
APIs	Connect POS systems and dashboards to centralized cloud services.
POS Systems	Record sales and inventory transactions at stores.
Management Dashboard	Allows administrators to monitor inventory and sales.
API Gateway	Controls and secures API requests.
VPN and Firewall	Protect communication and restrict unauthorized network access.

4. Working Process

1. A customer purchases a product through the store's POS system.
2. The POS system records the transaction.
3. The transaction is sent to the cloud through a secure API.
4. The API Gateway validates and routes the request.
5. Cloud services update the inventory database.
6. Transaction information is stored in cloud storage.
7. Analytics services process historical sales information.
8. Demand forecasting analyzes sales trends and predicts future requirements.
9. The management dashboard displays updated inventory and analytical results.
10. Security mechanisms protect the communication and stored information.

5. Cloud Service Models

The system demonstrates all three major cloud service models:

- **IaaS:** Virtual infrastructure, networking, and computing resources.
- **PaaS:** Analytics, forecasting, database, and application platform services.
- **SaaS:** The inventory dashboard and cloud-based inventory application accessed by users.

6. Security

Security is maintained through:

- **VPN:** Provides secure communication between stores and cloud resources.
- **Firewall:** Prevents unauthorized network access.
- **API Gateway:** Controls and monitors API requests.
- **Authentication:** Ensures only authorized users and devices access services.
- **Encryption:** Protects sensitive information during transmission and storage.
- **Access control:** Restricts inventory and administrative functions according to user roles.

7. Benefits

- Real-time inventory monitoring.
- Centralized data management.
- Easy scalability during high sales periods.
- Reduced infrastructure maintenance.
- Improved demand forecasting.
- Better decision-making using analytics.
- Secure communication between stores and cloud services.
- High availability and accessibility.

8. Conclusion

The Smart Retail Inventory Management System demonstrates how **IaaS, PaaS, SaaS, cloud storage, APIs, analytics, and security services** can be integrated into a single cloud-based solution. It helps retailers improve inventory management, forecast demand, and make faster business decisions.

2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing.

Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices.

Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

1. Introduction

A Cloud-Based Document Collaboration System is an online platform similar to Google Docs that allows multiple users to create, edit, share, and manage documents through web browsers.

Documents are stored in distributed cloud storage, while version control maintains previous versions of files. Web APIs synchronize document changes across different users and devices.

2. Objectives

The main objectives are:

- To allow users to access documents from anywhere.
- To support real-time collaborative editing.
- To store documents securely in cloud storage.
- To maintain different versions of documents.
- To synchronize changes across multiple devices.
- To provide document sharing and access permissions.
- To provide high availability and low-latency access.
- To protect documents using encryption and identity management.

3. Cloud Computing Components

Component	Role
Web Browser	Provides access to the collaboration platform.
Cloud Application	Provides document creation and editing functionality.
Cloud Storage	Stores documents and their versions.

Component	Role
Web APIs	Synchronize document updates between clients and cloud services.
Distributed Storage	Provides scalability and availability.
Version Control	Maintains previous versions of documents.
Identity Management	Authenticates users.
Access Control	Determines who can view or edit documents.
Network Infrastructure	Provides global connectivity and low latency.

4. Working Process

1. A user logs into the document collaboration platform through a browser.
2. The identity management service authenticates the user.

3. The user creates or opens a document.
4. The document is retrieved from distributed cloud storage.
5. Multiple users can access the same document.
6. When a user makes an edit, the change is sent to the cloud through Web APIs.
7. The cloud service synchronizes the changes with other connected users.
8. Updated versions are stored automatically.
9. Version control allows users to view or restore earlier versions.
10. Access permissions determine whether users can view, comment, or edit documents.

5. Cloud Service Models

- **IaaS:** Cloud networking, servers, storage infrastructure, and computing resources.
- **PaaS:** Application hosting, databases, APIs, synchronization, and collaboration services.
- **SaaS:** The complete document collaboration application accessed through a browser.

6. Security

The system uses multiple security mechanisms:

- **Encryption:** Protects documents and communication.
- **Identity management:** Authenticates users.
- **Access permissions:** Controls document access.
- **Authentication:** Prevents unauthorized users from accessing accounts.
- **Secure APIs:** Protect synchronization between clients and cloud services.
- **Version control:** Helps recover previous document states.

7. Benefits

- Real-time collaboration.
- Access from multiple devices.
- Automatic document synchronization.
- Centralized document storage.
- Version history and recovery.
- Reduced need for local storage.

- Global accessibility.
- High availability.
- Easy sharing and permission management.

8. Conclusion

The Cloud-Based Document Collaboration System demonstrates the use of cloud storage, SaaS, APIs, distributed infrastructure, version control, and identity management to provide real-time document collaboration. Cloud computing makes the system accessible, scalable, reliable, and suitable for organizations with geographically distributed users.

3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients.

Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

1. Introduction

An Online Food Ordering Platform is a cloud-based application that allows customers to browse restaurants and menus, place food orders, make payments, and track their orders using web browsers or mobile applications.

The frontend communicates with backend services using RESTful Web APIs. Cloud storage stores menu images, invoices, and customer-related data. Virtual machines and managed Kubernetes provide scalability and reliability.

2. Objectives

The objectives of the system are:

- To provide online food ordering through web and mobile clients.
- To connect frontend applications with backend services.
- To securely store customer and order information.
- To store menu images and invoices.
- To support a large number of users.
- To provide scalable cloud infrastructure.
- To ensure secure communication using HTTPS.
- To protect resources using authentication and role-based access control.

3. Cloud Computing Components

Component	Role
Web Client	Allows customers to order food through browsers.
Mobile Client	Provides food ordering through smartphones.

Component	Role
RESTful Web APIs	Connect frontend clients with backend services.
Cloud Platform	Hosts the backend application and services.
Virtual Machines	Provide computing resources for application services.
Managed Kubernetes	Manages containers and provides scalability.
Cloud Storage	Stores menu images, invoices, and customer data.
Authentication Tokens	Verify user identity.
HTTPS	Secures communication between clients and servers.
RBAC	Provides permissions based on user roles.

4. Working Process

1. The customer opens the food ordering application through a browser or mobile device.
2. The user logs in using their credentials.
3. The authentication service generates an authentication token.
4. The customer browses restaurant menus.
5. Menu information and images are retrieved from cloud services.
6. The customer selects food items and places an order.
7. The frontend sends the order through a RESTful Web API.
8. Backend services process the order.
9. Invoice and relevant information are stored securely in cloud storage.
10. Virtual machines and Kubernetes provide the required computing resources.
11. Kubernetes can scale application services when the number of users increases.

12.Role-Based Access Control ensures customers, restaurant staff, and administrators have appropriate permissions.

5. Cloud Service Models

IaaS:

- Virtual machines
- Virtual networks
- Computing infrastructure

PaaS:

- Managed Kubernetes
- Cloud application hosting
- Backend services
- Database and storage services

SaaS:

- Complete online food ordering application accessed by customers through web and mobile clients.

6. Security

The platform uses:

- **Authentication Tokens:** Verify the identity of users.

- **HTTPS:** Encrypts communication between clients and cloud services.
- **Role-Based Access Control:** Gives different permissions to customers, restaurant staff, delivery personnel, and administrators.
- **Encryption:** Protects sensitive information.
- **Secure Cloud Storage:** Protects invoices, images, and customer information.
- **API Security:** Prevents unauthorized access to backend services.

7. Scalability and Reliability

Managed Kubernetes allows the application to run multiple instances of backend services.

For example:

Normal Traffic



2 Application Instances



Users receive service normally

High Traffic



Kubernetes detects increased demand



More Application Instances



Improved Performance

This allows the system to handle sudden increases in customers during lunch hours, dinner hours, weekends, or promotional periods.

Virtual machines provide computing resources, while Kubernetes manages application containers and helps maintain service availability.

8. Benefits

- Food ordering from anywhere.
- Supports both web and mobile clients.
- Scalable during high traffic.
- Secure communication.

- Centralized cloud storage.
- High availability.
- Easy application deployment and management.
- Role-based security.
- Better reliability through managed infrastructure.

9. Conclusion

The Online Food Ordering Platform demonstrates the integration of cloud infrastructure, RESTful APIs, cloud storage, virtual machines, managed Kubernetes, authentication, HTTPS, and RBAC. Cloud computing allows the startup to provide a scalable and reliable food ordering service without requiring large amounts of physical infrastructure.